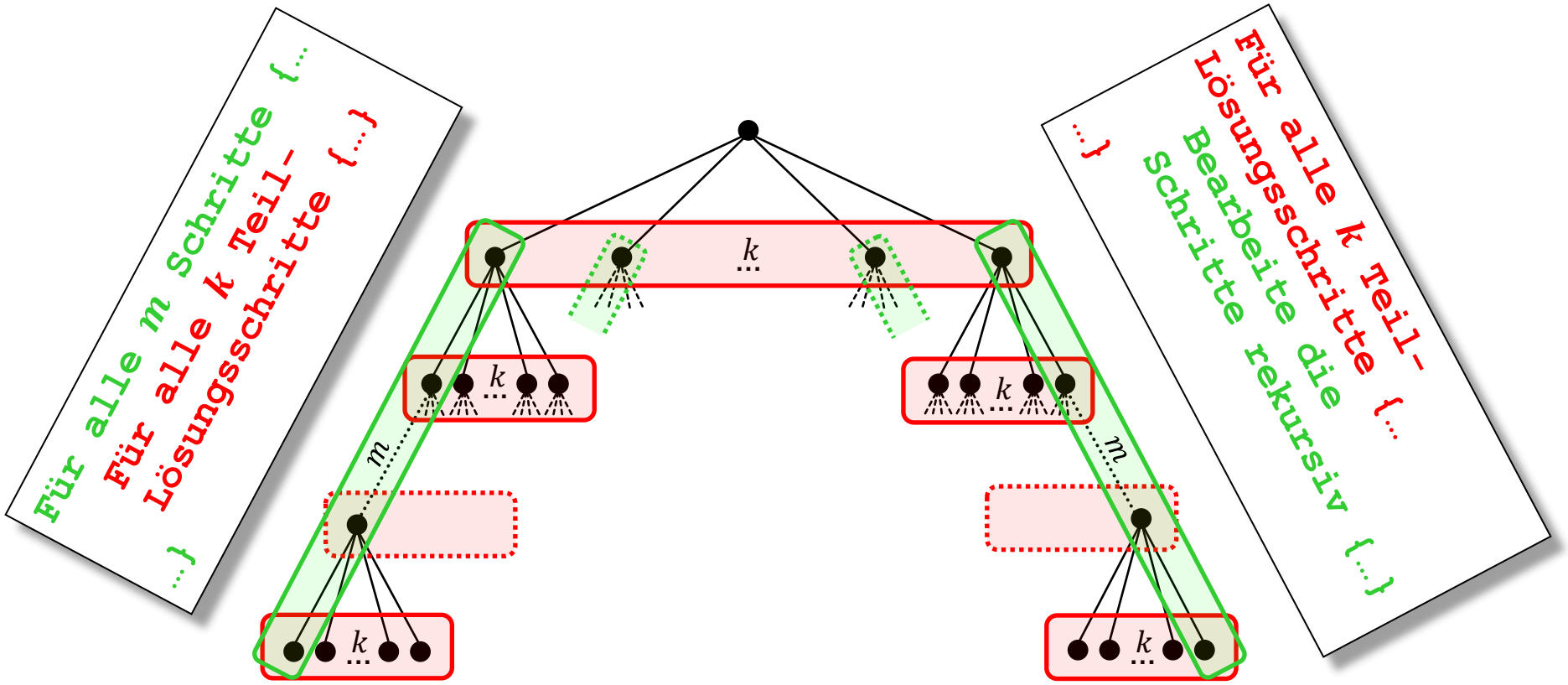
Rekursives vs. iteratives Backtracking

- Iteratives Backtracking
- Rekursives Backtracking



Algorithmus (iterativ)

```
bool threat(vector x, int j, int i)
{  // Wird Feld x[j] durch x[i] bedroht?
   return (x[i]==x[j] || abs(x[i]-x[j])==j-i);
}
```

§5.2 Das n-Damenproblem

```
int main() {
   int n,i,noSol=0;
   vector x; x.push_back(-1);
   while (!x.empty()) {                  // Iteratives Backtracking
      int  i      = x.size()-1;          // i-te Zeile i
      bool valid = false;                // valid≙unbedrohtes Feld
      while (!valid && x[i]<n-1) {       // Suche unbedrohtes Feld
         x[i]++; valid = true;
         for (j=0; j<i; j++) if ( threat(x,i,j) ) valid=false;
      }
      if (valid) {                       // Werte Zeile i aus.
         if (i==n-1) {                   // Lösung gefunden
            writeSolution(x); noSol++;
         } else x.push_back(-1);         // Vorwärts  suchen
      } else x.pop_back();               // Rückwärts suchen
   }
   return 0;
}
```

Fügt neues Element mit Wert **−1** an den Vektor an.

Algorithmus (rekursiv)

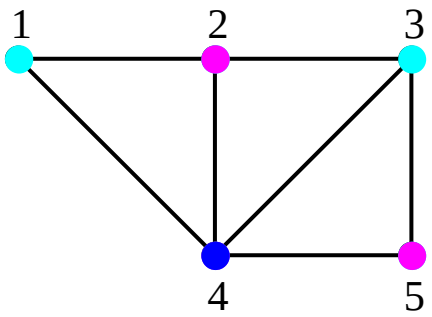
Initialisierung: `farbe[1..n]={0,...,0};`

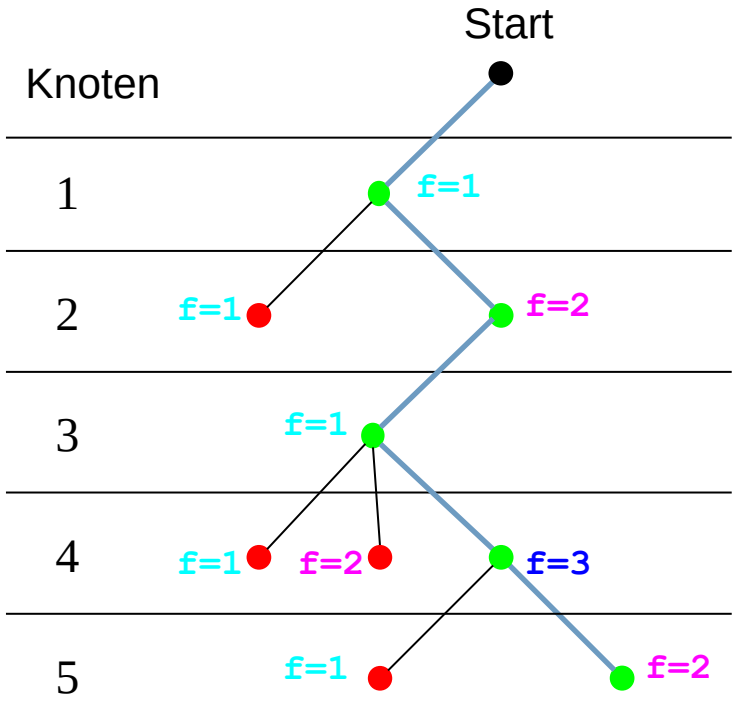
```
bool färbung(Graph G, int c, int i)      // G =(V,E)
{  // Vergibt Farbe für Knoten i, wenn 1..i-1 gefärbt sind.
   int  f = 0;                           // Farben 1,…,c
   bool result;
   do {
      f++; result = false;
      if (Farbe f ist möglich für Knoten i) {
         farbe[i] = f;
         if (i < n) {
            result = färbung(G, c, i+1); // Färben des Sub-Trees
            if (!result) farbe[i] = 0;   // Backtracking
         } else result = true;           // Gültige Färbung…
      }                                  // …gefunden
   } while ( !result && f!=c );
   return result;
}
```

§5.3 Färbung von Graphen

Beispiel:

- Ist eine 3-Färung (●,●,●) möglich?



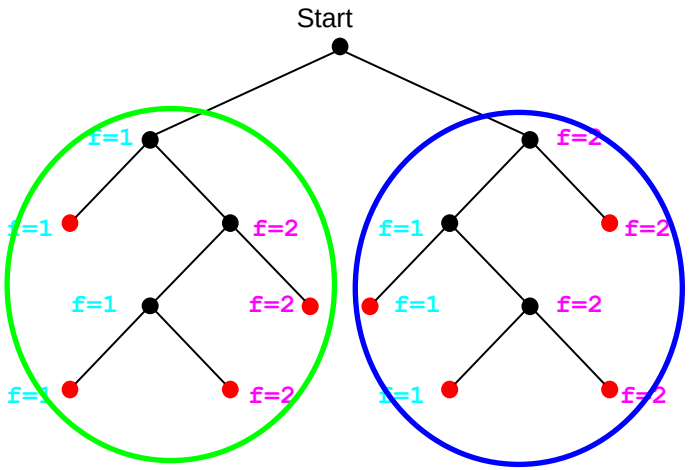


● Gültige Färbung ● Sackgasse

- **Antwort:**
  Ja, es gibt einen gültig gefärbten Pfad von der Wurzel zu einem Blatt.

Beobachtung (speziell)

- Der **linke Teilbaum** ist der Versuch einer 2-Färbung beginnend mit Farbe 1 im ersten Knoten.
- Der **rechte Teilbaum** ist der Versuch einer 2-Färbung beginnend mit Farbe 2 im ersten Knoten.



- Diese beiden Teilbäume entstehen auseinander durch Vertauschung der Farben.
- Die Berechnung des rechten Teilbaumes ist überflüssig, weil aus dem linken Teilbaum bereits ersichtlich ist, dass es kein 2-Färbung gibt.
- Das gilt auch, wenn man mit Farbe 2 in Knoten 1 beginnt!

Algorithmus

Aufruf: `färbung(G, c, 1, 1);`

```
bool färbung(Graph G, int c, int i, int F ) {
   int  f = 0;
   bool result;
   do {
      f++; result = false;
      if (Farbe f ist möglich für Knoten i) {
         farbe[i] = f;
         if (i < n) {
            if (f==F) result = färbung(G, c, i+1, F+1);
            else      result = färbung(G, c, i+1, F  );
            if (!result) farbe[i] = 0;
         } else result = true;
      }
   } while ( !result && f!=c && f!=F );
   return result;
}
```